

OSeMOSYS Project Code Explained

EN401 - Energy Systems Modelling

Section	Contents
1. Project Structure	Directory layout and file roles
2. run_osemosys.py	Python runner script — line-by-line walkthrough
3. .dat File Format	GNU MathProg data file syntax and structure
4. OSeHO3.dat	Hands-on 3 data file — sets, params, demand
5. OSeHO4.dat — additions	New parameters added for fossil fuel supply
6. OSeHO5.dat — additions	Power plant and grid parameters
7. OSeHO6.dat — additions	Hydropower parameters and cost comparison
8. Key Parameters Reference	All OSeMOSYS parameters used, defined

Language: Python 3 (runner) | GNU MathProg (data files) | GLPK v5.0 (solver)
Generated: September 29, 2026

Chapter 1: Project Structure

The OSeMOSYS project is organised as follows. Each file has a distinct role in the modelling pipeline:

```
SEM5 pros/
■■■ run_osemosys.py          # Python script: runs the full pipeline
■■■ .venv/                  # Python virtual environment
■   ■■■ bin/otoole          # otoole toolkit (format conversion)
■■■ osemosys_project/
■   ■■■ models/
■   ■   ■■■ osemosys_fast.txt # OSeMOSYS model: LP equations (MathProg)
■   ■   ■■■ osemosys.txt     # Standard OSeMOSYS formulation
■   ■■■ data/
■   ■   ■■■ simplicity.dat   # Demo/test dataset
■   ■■■ results/            # Output CSVs from solver
■   ■■■ config.yaml         # otoole conversion config
■■■ EN401/
    ■■■ H03/
    ■   ■■■ OSeH03.dat       # Hands-on 3 input data (MathProg format)
    ■   ■■■ OSeH03_solution.txt # GLPK solution output
    ■   ■■■ Hands_on_3.pdf   # Assignment handout
    ■■■ H04/ (same structure)
    ■■■ H05/ (same structure)
    ■■■ H06/ (same structure)
```

File Type Roles

Extension	Language	Role
.py	Python 3	Automation: argument parsing, subprocess calls, output display
.txt (model)	GNU MathProg	OSeMOSYS model: LP constraint equations and objective function
.dat	GNU MathProg	Data files: sets and parameter values fed to the model
.txt (solution)	GLPK output	Solver output: variable values, constraint activity, objective
.yaml	YAML	otoole config: column names and types for CSV conversion
.csv	CSV	Result tables output by GLPK (one file per variable)

Chapter 2: run_osemosys.py — Python Runner Script

This is the main Python automation script. It ties together the three tools (otoole for conversion, glpsol for solving, and pandas for result display) into a single command-line interface. Total: 182 lines.

Section 1: Imports and Constants (Lines 1–22)

Code	What It Does
<code>#!/usr/bin/env python3</code>	Shebang: tells the OS to use Python 3
<code>import argparse</code>	Parses command-line flags (--data, --model, etc.)
<code>import os</code>	OS-level path operations
<code>import re</code>	Regex: extracts objective value from solution text
<code>import subprocess</code>	Runs external programs: glpsol, otoole
<code>import sys</code>	<code>sys.exit()</code> on errors; <code>sys.executable</code> for <code>.venv</code> path
<code>import time</code>	Times the solver run
<code>from pathlib import Path</code>	Modern path handling (cross-platform)
<code>import pandas as pd</code>	Reads result CSVs for the summary display
<code>BASE_DIR = Path(__file__).resolve().parent</code>	Directory where <code>run_osemosys.py</code> lives
<code>PROJECT_DIR = BASE_DIR / 'osemosys_project'</code>	Joins paths using <code>/</code> operator
<code>DEFAULT_MODEL = PROJECT_DIR / 'models' / 'osemosys LP model file'</code>	Default LP model file
<code>DEFAULT_RESULTS = PROJECT_DIR / 'results'</code>	Default output folder

Section 2: ensure_glpsol() (Lines 24–29)

```
def ensure_glpsol():
    result = subprocess.run(['which', 'glpsol'], capture_output=True, text=True)
    if result.returncode != 0:
        print('[ERROR] glpsol not found in PATH.', file=sys.stderr)
        sys.exit(1)
```

Purpose: Verifies that GLPK is installed before attempting to solve. `subprocess.run(['which', 'glpsol'])` asks the shell to find the `glpsol` binary. If it returns a non-zero exit code (i.e., not found), the script prints an error and exits immediately rather than failing later with a cryptic message.

Section 3: convert_to_datafile() (Lines 31–51)

Code	What It Does
<code>def convert_to_datafile(input_path, config_path):</code>	Converts Excel or CSV to MathProg .dat format using otoole
<code> input_format = 'excel' if ... else 'csv'</code>	Determines format from file extension (.xlsx vs directory)
<code> otoole_bin = Path(sys.executable).parent / 'otoole'</code>	Finds otoole inside the same .venv as the current Python
<code> cmd = [otoole_bin, 'convert', input_format, ...]</code>	Builds the file command and: converts <code>input_format</code> -> <code>datafile</code>
<code> res = subprocess.run(cmd, capture_output=True, text=True)</code>	Runs <code>otoole</code> as subprocess, capturing stdout/stderr

Code	What It Does
<pre>if res.returncode != 0: sys.exit(res.returncode)</pre>	Propagates otool errors — exit code passes through to shell

The otool 'convert' command reads either an Excel workbook (one sheet per parameter) or a folder of CSVs and writes a single .dat

Section 4: run_glpsol() (Lines 53–92)

Code	What It Does
<pre>results_dir.mkdir(parents=True, exist_ok=True)</pre>	Creates the results folder if it doesn't exist (no error if it does)
<pre>solution_file = results_dir / f'{data_path.stem}.OSeHO3_solution.txt'</pre>	Creates solution file OSeHO3_solution.txt (stem = name without extension)
<pre>cmd = ['glpsol', '-m', model_path, '-d', data_path, '-o', solution_file]</pre>	Command to run glpsol on model file -d: data file -o: file to write solution to this file
<pre>proc = subprocess.run(cmd, cwd=str(PROJECT_DIR))</pre>	Runs from osemosys_project/ so relative ResultsPath works
<pre>elapsed = time.time() - start_time</pre>	Wall-clock time of the entire solve (usually < 0.1 seconds)
<pre>if 'Objective:' in line: match = re.search(r'RegExp', line)</pre>	Regex extracts the objective value from the solution text header

The core glpsol command is: **glpsol -m osemosys_fast.txt -d OSeHO3.dat -o solution.txt**. This is equivalent to what you would type manually in a terminal. The script just automates it, captures errors, and extracts the objective value.

Section 5: display_summary() (Lines 94–133)

Code	What It Does
<pre>cost_file = results_dir / 'TotalDiscountedCosts.csv'</pre>	Looks for this specific result CSV after solving
<pre>df_cost = pd.read_csv(cost_file)</pre>	Reads it into a pandas DataFrame for easy manipulation
<pre>df_sorted = df_cost.sort_values(by='YEAR')</pre>	Sorts rows by year for chronological display
<pre>pd.concat([df_sorted.head(3), df_sorted.tail(3)])</pre>	Shows first 3 and last 3 years as a preview
<pre>df_cap = pd.read_csv(cap_file)</pre>	Reads TotalCapacityAnnual.csv to show installed capacity
<pre>df_latest = df_cap[df_cap['YEAR'] == latest_year]</pre>	Filters to the final year only
<pre>top_techs = df_latest[df_latest['VALUE'] > 0.01].head(5)</pre>	Shows top 5 technologies with > 0.01 units capacity

Section 6: main() — CLI and Orchestration (Lines 135–181)

```
# Example usage:
# 1. Run with .dat file directly:
.venv/bin/python run_osemosys.py --data EN401/H03/OSeH03.dat

# 2. Run with Excel input (auto-converts):
.venv/bin/python run_osemosys.py --excel data_ho/OSeH03.xlsx

# 3. Specify a custom model and results folder:
.venv/bin/python run_osemosys.py --data OSeH06.dat --model osemosys.txt --results out/
```

Code	What It Does
<pre>parser = argparse.ArgumentParser(...)</pre>	Creates a CLI argument parser
<pre>parser.add_argument('--data', '-d', ...)</pre>	-d or --data: path to .dat file (default: simplicity.dat)

Code	What It Does
<code>parser.add_argument('--excel', '-x', ...)</code>	-x or --excel: path to .xlsx; triggers otoole conversion first
<code>parser.add_argument('--model', '-m', ...)</code>	-m or --model: override the model file (default: osemosys_fast.txt)
<code>ensure_glpсол()</code>	First check: abort immediately if GLPK not found
<code>if args.excel: convert_to_datafile(...)</code>	Branch: if Excel input, convert to .dat first, then solve
<code>sol_file, obj_val, elapsed = run_glpсол(...)</code>	Runs the solver, returns solution path + objective + time
<code>display_summary(results_dir, obj_val)</code>	Prints the formatted summary to console

Chapter 3: GNU MathProg Data File (.dat) Format

The .dat files are not Python or YAML — they are written in GNU MathProg Data Format, the language understood by GLPK. They contain only data (no equations). The equations live in osemosys_fast.txt. The two files are combined at solve time.

Syntax Rules

Syntax Element	Meaning	Example
set NAME := val1 val2 ... ;	Declares a set (list of labels)	set YEAR := 2021 2022 2023 ;
param NAME := key value ... ;	Declares a scalar or indexed parameter	param DiscountRate := 0.1 ;
param default D : NAME :=	Sets D as the default for unlisted entries; only one default is allowed	param default 0 : CapacityFactor :=
REGION TECH YEAR VALUE	Multi-index parameter entry (space-separated)	SC_0 PWRDSL 2021 1200.0
# comment	Line comment — ignored by parser	# OSeMOSYS data file - Hands-on 3
;	Terminates every set or param declaration	SC_0 BACKSTOP 2035 99999.0 ;

Critical Design Pattern: param default D

```
# This means: CapacityFactor = 1.0 for ALL region/tech/timeslice/year combinations
# UNLESS a specific override is listed below.
param default 1 : CapacityFactor :=
;

# Contrast with a parameter that has specific values:
param default 0 : SpecifiedAnnualDemand :=
SC_0 ELC003 2021 20.0    # Explicit: 20 PJ in 2021
SC_0 ELC003 2022 25.0    # Explicit: 25 PJ in 2022
; # All other region/fuel/year combos default to 0
```

This pattern is why the .dat files can be concise: you only write entries that differ from the default. Every unspecified [region, technology, year] triple automatically gets the default value.

Timeslices Explained

OSeMOSYS splits each year into intra-annual time segments called timeslices. These represent different seasons and times of day. In our model:

Timeslice Code	Meaning	YearSplit (fraction of year)	Demand profile share
RD	Rainy season, Daytime	0.208 (20.8% of year)	25% of annual demand
RN	Rainy season, Night	0.208 (20.8% of year)	20% of annual demand
DD	Dry season, Daytime	0.292 (29.2% of year)	30% of annual demand
DN	Dry season, Night	0.292 (29.2% of year)	25% of annual demand

Note: $0.208 + 0.208 + 0.292 + 0.292 = 1.0$ (the four slices cover exactly one year). Demand profile shares also sum to 1.0 ($0.25 + 0.20 + 0.30 + 0.25 = 1.0$).

Chapter 4: OSeHO3.dat — Hands-on 3 Data File

This is the first solved model. It defines only 2 technologies (MINBACK, BACKSTOP) and 2 fuels (BACK, ELC003). Total: 394 lines.

Sets Declaration (Lines 3–13)

Code	What It Does
set YEAR := 2021 2022 ... 2035 ;	Planning horizon: 15 years. Every parameter indexed by year must use these values.
set REGION := SC_0 ;	Single region model. SC_0 = 'Small Country 0' (pedagogical label).
set TECHNOLOGY := MINBACK BACKSTOP ;	Only 2 technologies in HO3. All parameters indexed by TECHNOLOGY must use these values.
set FUEL := BACK ELC003 ;	BACK = virtual backstop fuel. ELC003 = electricity commodity (LHV delivered).
set TIMESLICE := RD RN DD DN ;	4 intra-annual time segments (rainy/dry x day/night).
set MODE_OF_OPERATION := 1 ;	Mode 1 only — no dual-mode technologies in this model.
set EMISSION := ;	Empty: no emissions tracking in HO3.
set STORAGE := ;	Empty: no storage technologies.

YearSplit Parameter (Lines 15–76)

```
param default 0 : YearSplit :=
RD 2021 0.208 # Rainy Daytime = 20.8% of the year
RN 2021 0.208 # Rainy Night = 20.8% of the year
DD 2021 0.292 # Dry Daytime = 29.2% of the year
DN 2021 0.292 # Dry Night = 29.2% of the year
... (same for all years 2022-2035)
;
```

YearSplit[timeslice, year]: the fraction of a year that each timeslice represents. The model uses this to compute annual energy from per-timeslice activity variables: $\text{AnnualEnergy} = \text{sum over timeslices of } (\text{Activity}[\text{ts}] * \text{YearSplit}[\text{ts}, \text{y}])$.

DiscountRate and Demand (Lines 77–97)

Code	What It Does
param default 0.1 : DiscountRate := ;	10% annual discount rate for ALL regions (default, no overrides needed). Used to compute NPV.
param default 0 : SpecifiedAnnualDemand :=	Annual electricity demand in PJ. Default 0 (no demand) unless specified.
SC_0 ELC003 2021 20.0	Region SC_0, fuel ELC003, year 2021: demand = 20 PJ
SC_0 ELC003 2022 25.0	Year 2022: demand = 25 PJ (+5 PJ/year linear growth)
SC_0 ELC003 2035 90.0	Year 2035: demand = 90 PJ (final year, 4.5x initial demand)

SpecifiedDemandProfile (Lines 98–159)

```

param default 0 : SpecifiedDemandProfile :=
SC_0 ELC003 RD 2021 0.25 # 25% of annual demand in Rainy Daytime
SC_0 ELC003 RN 2021 0.20 # 20% of annual demand in Rainy Night
SC_0 ELC003 DD 2021 0.30 # 30% of annual demand in Dry Daytime
SC_0 ELC003 DN 2021 0.25 # 25% of annual demand in Dry Night
; # (same profile for all years 2022-2035)

```

SpecifiedDemandProfile[region, fuel, timeslice, year]: distributes annual demand across timeslices. Values must sum to 1.0 per year. The LP uses this to compute the demand that must be met in each timeslice: $\text{RateOfDemand} = \text{SpecifiedAnnualDemand} * \text{Profile} / \text{YearSplit}$.

Cost Parameters: CapitalCost, FixedCost, VariableCost (Lines 160–255)

Code	What It Does
param default 0 : CapitalCost :=	Capital cost (\$/kW or M\$/GW) — paid once when capacity is built.
SC_0 MINBACK 2021 99999.0	Prohibitively high CAPEX for MINBACK — discourages investment in it.
SC_0 BACKSTOP 2021 99999.0	Same for BACKSTOP — the model uses it only as a last resort.
param default 0 : FixedCost :=	Annual fixed O&M cost (\$/kW/year) — paid every year regardless of output.
SC_0 MINBACK 2021 99999.0	Also prohibitively high — reinforces that backstop is expensive.
param default 0 : VariableCost :=	Variable cost (\$/PJ) — paid per unit of energy produced.
SC_0 MINBACK 1 2021 999.0	Index: [region, technology, mode, year]. 999 \$/PJ variable cost.
SC_0 BACKSTOP 1 2021 999.0	Both backstop techs have identical, very high variable costs.

Activity Ratios: InputActivityRatio and OutputActivityRatio (Lines 268–316)

These are the most important parameters — they define the energy flow through each technology:

```

# MINBACK: produces BACK fuel from nothing (source/mine)
param default 0 : OutputActivityRatio :=
SC_0 MINBACK BACK 1 2021 1.0 # For every 1 unit of activity, output 1 PJ of BACK

# BACKSTOP: consumes BACK fuel, outputs ELC003 electricity
param default 0 : InputActivityRatio :=
SC_0 BACKSTOP BACK 1 2021 1.0 # Input: 1 PJ of BACK per unit of activity

param default 0 : OutputActivityRatio :=
SC_0 BACKSTOP ELC003 1 2021 1.0 # Output: 1 PJ of ELC003 per unit of activity

# Energy flow diagram:
# [MINBACK] --- (BACK fuel) ---> [BACKSTOP] --- (ELC003) ---> [Consumer demand]

```

The ratio of Input to Output defines **efficiency**. Here both are 1.0, so $\text{efficiency} = \text{Output}/\text{Input} = 1.0 = 100\%$ (a virtual/lossless backstop). In HO5, PWRDSL has $\text{Input}(\text{DSL})=3.0$ and $\text{Output}(\text{ELC})=1.0$ — representing ~33% thermal efficiency for a diesel generator.

CapacityToActivityUnit (Line 256–259)

```
param default 1 : CapacityToActivityUnit :=
SC_0 MINBACK 1.0      # 1 GW capacity = 1 PJ/yr activity (fuel supply)
SC_0 BACKSTOP 31.536  # 1 GW capacity = 31.536 PJ/yr activity (power plant)
;

# Why 31.536? Because 1 GW running continuously for 1 year produces:
# 1 GW * 8760 hours/year * 3600 seconds/hour = 31,536,000 MJ = 31.536 PJ
```

Chapter 5: OSeHO4.dat — What Changed from HO3

HO4 adds fossil fuel supply technologies. Only the differences from HO3 are highlighted. The file grows from 394 lines to ~530 lines (additional parameter blocks for MINNGS and IMPDSL).

Updated Technology Set

```
# H03:
set TECHNOLOGY := MINBACK BACKSTOP ;

# H04 (adds MINNGS and IMPDSL):
set TECHNOLOGY := MINBACK BACKSTOP MINNGS IMPDSL ;

# Also adds a new fuel:
set FUEL := BACK ELC003 NGS DSL ;
#           ^^^ ^^^
#           Natural gas   Diesel fuel
```

New Technology Parameters

Code	What It Does
SC_0 MINNGS 2021 0.0 (CapitalCost)	Gas mining has zero capital cost — the resource is in-ground, no plant needed
SC_0 MINNGS 1 2021 2.5 (VariableCost)	Variable cost = 2.5 \$/PJ — cheap to extract compared to backstop (999)
SC_0 MINNGS NGS 1 2021 1.0 (OutputActivityRatio)	MINNGS produces 1 PJ of NGS per unit of activity
SC_0 IMPDSL 2021 0.0 (CapitalCost)	Diesel import has zero capital cost — no refinery, just import
SC_0 IMPDSL 1 2021 15.0 (VariableCost)	Variable cost = 15 \$/PJ — more expensive than gas, but affordable
SC_0 IMPDSL DSL 1 2021 1.0 (OutputActivityRatio)	IMPDSL produces 1 PJ of DSL per unit of activity

WHY THE COST DOESN'T CHANGE: Although MINNGS and IMPDSL are now defined and can produce NGS and DSL fuels, there

Chapter 6: OSeHO5.dat — Power Plants and Grid

HO5 is where the model gets real. Four new technologies are added that complete the supply chain: PWRDSL, PWRNGS (generators), PWRTRN (transmission), PWRDIST (distribution). File grows to ~900 lines.

New Technologies and Their Costs

```
set TECHNOLOGY := MINBACK BACKSTOP MINNGS IMPDSL PWRDSL PWRNGS PWRTRN PWRDIST ;
set FUEL        := BACK ELC003 NGS DSL ELCHV ;
#
#                      ^^^^
#                      High-voltage electricity (pre-distribution)
```

Capital Costs — Power Plants and Grid (\$/kW)

Technology	CapitalCost (\$/kW)	FixedCost (\$/kW/yr)	VariableCost (\$/PJ)	OperationalLife
PWRDSL (diesel plant)	1,200	35	0.6	20 years
PWRNGS (gas turbine)	1,200	35	1.4	20 years
PWRTRN (transmission)	700	7	1.5	40 years
PWRDIST (distribution)	1,500	15	1.5	40 years

Activity Ratios — The Complete Supply Chain

```
# PWRDSL: consumes DSL (diesel), outputs ELCHV (high-voltage electricity)
InputActivityRatio: SC_0 PWRDSL DSL 1 2021 3.0 # 3 PJ diesel in
OutputActivityRatio: SC_0 PWRDSL ELCHV 1 2021 1.0 # 1 PJ electricity out
# Efficiency = 1/3 = 33% (realistic for diesel generators)

# PWRNGS: consumes NGS (gas), outputs ELCHV
InputActivityRatio: SC_0 PWRNGS NGS 1 2021 2.0 # 2 PJ gas in
OutputActivityRatio: SC_0 PWRNGS ELCHV 1 2021 1.0 # 1 PJ electricity out
# Efficiency = 1/2 = 50% (gas turbines are more efficient than diesel)

# PWRTRN: transmits ELCHV, outputs ELCHV (with 5% transmission loss)
InputActivityRatio: SC_0 PWRTRN ELCHV 1 2021 1.05
OutputActivityRatio: SC_0 PWRTRN ELCHV 1 2021 1.0

# PWRDIST: distributes ELCHV to final consumers as ELC003
InputActivityRatio: SC_0 PWRDIST ELCHV 1 2021 1.0
OutputActivityRatio: SC_0 PWRDIST ELC003 1 2021 1.0

# Full chain: MINNGS/IMPDSL -> PWRNGS/PWRDSL -> PWRTRN -> PWRDIST -> ELC003
```

CapacityToActivityUnit for Power Plants

```
param default 1 : CapacityToActivityUnit :=
SC_0 PWRDSL 31.536 # 1 GW * 8760 hrs * 3600 s = 31.536 PJ/yr
SC_0 PWRNGS 31.536
SC_0 PWRTRN 31.536
SC_0 PWRDIST 31.536
;
# This conversion factor links GW (capacity) to PJ/yr (energy activity)
# It appears in the capacity adequacy constraint:
# RateOfActivity[ts] * YearSplit[ts] <= TotalCapacity * CapacityFactor * CapacityToActivityUnit
```

Chapter 7: OSeHO6.dat — Hydropower Parameters

HO6 adds two new technologies: MINHYD (hydro resource extraction) and PWRHYD (hydroelectric plant). The key difference from fossil fuels is zero variable cost. File grows to ~1,300 lines.

New Technology Set

```
set TECHNOLOGY := MINBACK BACKSTOP MINNGS IMPDSL PWRDSL PWRNGS
                  PWRTRN PWRDIST MINHYD MINBIO PWRHYD PWRBIO ;
#                  ^^^^^^ ^^^^^^^^^^^^^^^^^^ ^^^^^^ ^^^^^^
#                  Hydro&Bio resource   Hydro plant   Bio plant

set FUEL := BACK ELC003 NGS DSL ELCHV HYD BIO ;
#                  ^^^  ^^^
#                  Hydro fuel   Biomass fuel
```

Hydro Cost Parameters — Key Differences vs Fossil

Parameter	PWRDSL (diesel)	PWRNGS (gas)	PWRHYD (hydro)
CapitalCost (\$/kW)	1,200	1,200	2,500 (2x more expensive to build)
FixedCost (\$/kW/yr)	35	35	25 (slightly cheaper to maintain)
VariableCost (\$/PJ)	0.6	1.4	0.0 (NO fuel cost — water is free)
OperationalLife (yr)	20	20	40 (lasts twice as long)
Input fuel	DSL	NGS	HYD (hydro water resource)
Efficiency	33%	50%	~85% (very high, no thermodynamic limit)

Activity Ratios for Hydro

```
# MINHYD: extracts hydro resource (water flow)
OutputActivityRatio: SC_0 MINHYD HYD 1 2021 1.0
# No input fuel needed — rainfall is exogenous

# PWRHYD: converts water flow to high-voltage electricity
InputActivityRatio: SC_0 PWRHYD HYD 1 2021 1.2 # 1.2 PJ water in
OutputActivityRatio: SC_0 PWRHYD ELCHV 1 2021 1.0 # 1.0 PJ electricity out
# Efficiency = 1/1.2 = 83%

# Then feeds into the same PWRTRN -> PWRDIST -> ELC003 chain
```

Why the LP Chooses Hydro

The LP minimises total **NPV cost** = sum over all years of (CapEx + FixedOpex + VariableOpex) / (1 + discount_rate)^t. For PWRHYD:

```

NPV_hydro = CapitalCost[2021] * NewCapacity[2021]           # Large upfront
              + sum_t( FixedCost[t] * TotalCapacity[t] / 1.1^t )   # Small annual
              + sum_t( VariableCost[t] * RateOfActivity[t] / 1.1^t ) # = 0 (free!)

NPV_diesel = CapitalCost[2021] * NewCapacity[2021]           # Half of hydro
              + sum_t( FixedCost[t] * TotalCapacity[t] / 1.1^t )
              + sum_t( 0.6 * FuelActivity[t] * 3.0_input_ratio / 1.1^t )# Fuel cost compounds!

# Over 40 years (hydro lifetime), fuel savings dominate over higher CAPEX
# LP correctly selects hydro as least-cost despite higher initial investment

```

Chapter 8: OSeMOSYS Parameters — Complete Reference

Every parameter declared in the .dat files and its mathematical role in the LP model:

Parameter	Index	Unit	Role in LP Model
YearSplit	[ts, y]	fraction	Weight of each timeslice within a year. Converts activity to annual
DiscountRate	[r]	fraction	Annual discount rate for NPV calculation. Default 0.1 = 10%.
SpecifiedAnnualDemand	[r, f, y]	PJ/yr	Total annual demand for fuel f in region r year y. Drives the LP con
SpecifiedDemandProfile	[r, f, ts, y]	fraction	Intra-annual demand distribution. Must sum to 1.0 per [r, f, y].
CapitalCost	[r, t, y]	M\$/GW	Overnight construction cost per GW of new capacity. Paid in the y
FixedCost	[r, t, y]	M\$/GW/yr	Annual fixed O&M cost per GW of installed capacity. Paid every ye
VariableCost	[r, t, m, y]	M\$/PJ	Operating cost per PJ of energy output. Includes fuel cost for supp
CapacityFactor	[r, t, ts, y]	fraction	Maximum fraction of capacity usable in each timeslice. 1.0 = alway
OperationalLife	[r, t]	years	Plant lifetime. Determines when residual capacity retires.
ResidualCapacity	[r, t, y]	GW	Pre-existing capacity inherited before the model start year.
InputActivityRatio	[r, t, f, m, y]	PJ_in/PJ_out	Fuel consumed per PJ of output. Inverse is efficiency.
OutputActivityRatio	[r, t, f, m, y]	PJ_out/PJ_in	Output of fuel f per unit of activity. Defines what a technology prod
CapacityToActivityUnit	[r, t]	PJ/(GW*yr)	Converts GW capacity to PJ/yr activity. For power plants: 31.536.
TotalAnnualMaxCapacity	[r, t, y]	GW	Hard upper limit on total installed capacity. -1 = unconstrained.
TotalAnnualMinCapacity	[r, t, y]	GW	Hard lower limit on total installed capacity. 0 = no minimum.
AvailabilityFactor	[r, t, y]	fraction	Fraction of year a plant is available (accounting for planned outage
ReserveMargin	[r, y]	fraction	Required capacity buffer above peak demand. 1.0 = no reserve re

LP Variable Summary

Variable	Index	Unit	Meaning
NewCapacity	[r, t, y]	GW	Decision var: how much new capacity to build in year y (what the LP o
TotalCapacityAnnual	[r, t, y]	GW	Cumulative installed capacity (NewCapacity + residual - retired)
RateOfActivity	[r, ts, t, m, y]	PJ/yr	Activity rate in each timeslice (dispatch decision)
ProductionByTechnology	[r, ts, t, f, y]	PJ	Energy produced by technology t of fuel f in timeslice ts
TotalAnnualTechnology...	[r, t, f, y]	PJ	Total annual production of fuel f by technology t
CapitalInvestment	[r, t, y]	M\$	Capital spend = NewCapacity * CapitalCost
DiscountedCapitalInvestment	[r, t, y]	M\$	Capital spend discounted to base year NPV
TotalDiscountedCost	[r, y]	M\$	Sum of all discounted costs — the objective function